Odofin Abdulalim Boluwatife

ABO30@aber.ac.uk

8th of May 2024

Main Individual Assignment: The Question Bank Project

CS12320 – Programming using object-oriented Language.

Contents

Introduction 3
Actors' descriptions 5
Use case description. 5
Design 6
Textual class description 6
Test Tables 10
SCREENSHOTS 13
EVALUATION 15
WHAT WAS DIFFICULT 16
WHAT REMAINS TO BE DONE 16
FLAIR 16
MARK 16
References 17

Introduction

This project endeavours to develop a Java-based, object-oriented program specially for a local school, focusing on creating a comprehensive quiz platform. Its primary goal is to empower students to assess their understanding and enhance exam preparation. The program features a user-friendly interface, serving as a seamless tool for both students and teachers. Teachers retain the authority to create modules, question banks, and individual questions, with the flexibility to modify them as needed. Conversely, students have access to quizzes associated with specific modules, where questions are presented in a randomized order for a good learning experience.

This program serves as a vital tool in mitigating human errors that can occur during manual processes. It effectively eliminates the risk of typos by the teachers when composing questions and prevents students from mistakenly selecting incorrect modules due to compilation errors. Additionally, it significantly reduces the time and effort required to create a comprehensive question bank, minimizing the likelihood of errors associated with manual composition.

Moreover, the program addresses issues of inflexibility that often accompany manual processes. Human errors, such as forgetting to delete outdated or irrelevant information when adding or removing questions, are effectively avoided. This ensures clarity and coherence in the examination process, preventing confusion and frustration for students.

Furthermore, the program enhances security measures by safeguarding against potential risks such as theft, damage, or leakage of written questions. With its digital infrastructure, the program provides a secure environment for quiz administration, ensuring smooth operations while offering a user-friendly interface for both teachers and students.

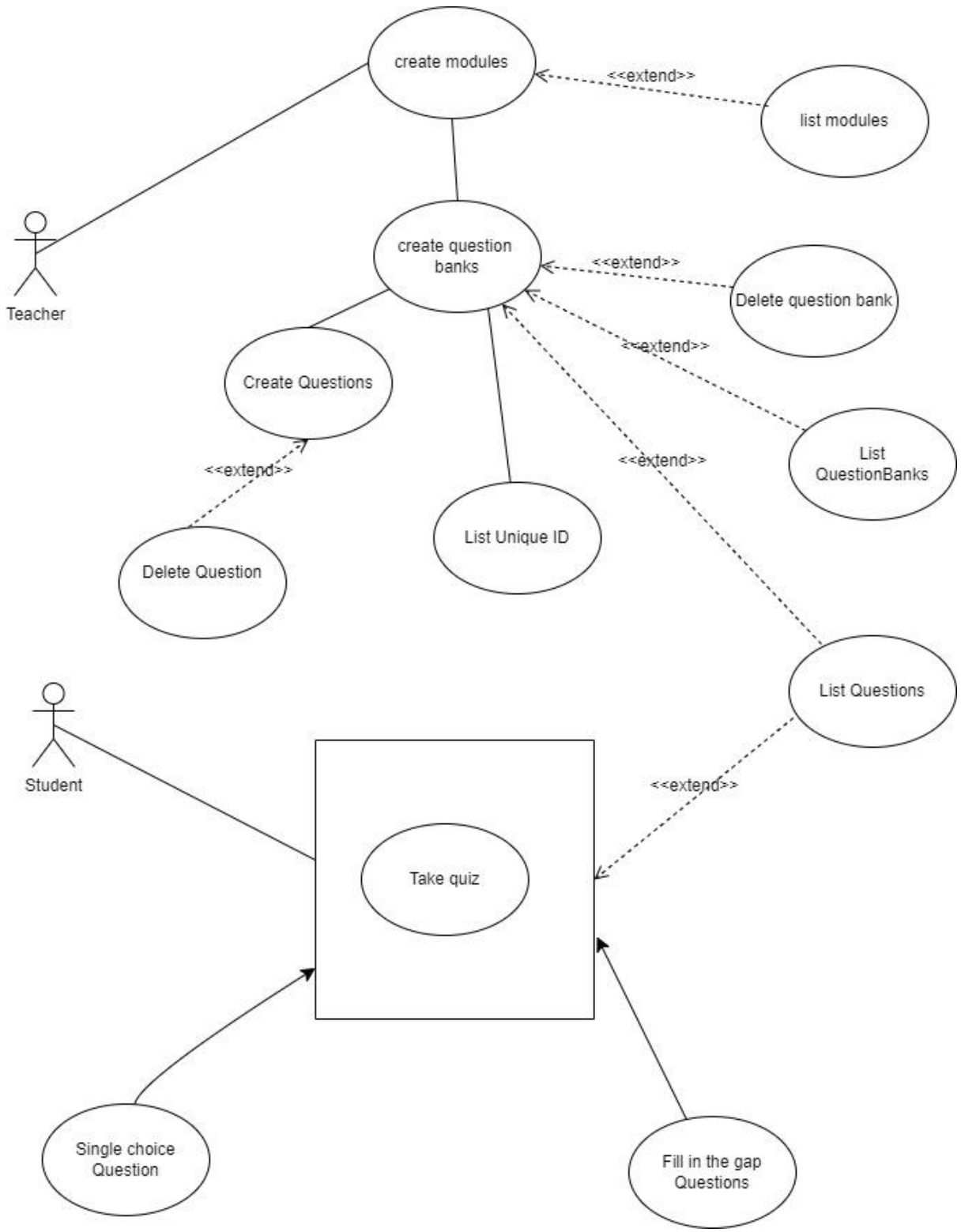


FIGURE 1USE CASE DIAGRAM

Use case diagram.

The image above is the use case diagram that illustrates the primary functionalities of the program and more like a skeletal structure of how the program is linked and works. It involves two major actors(users), which are.

Actors' descriptions

Teachers: they're the major actor for this program due to the amount of access they have. They're basically the foundation by having ground to create module which all other functions are being built upon.

Students: students on the other hand gets to take the quiz on any module they wanted.

Use case description.

Create Module: this feature empowers teachers to generate new modules for organizing question banks, which houses various questions. Importantly, this functionality imposes a restriction ensuring that module names are limited to a maximum of 7 characters. This constraint aims to maintain consistency and efficiency within the system, preventing excessively lengthy or ambiguous module names that could potentially hinder organization and navigation. By adhering to this restriction, teachers can swiftly create and easily identifiable modules, thereby enhancing the overall usability and effectiveness of the program.

Question Banks: it houses and stores different questions and allows access for both the teacher and the student in creating and storing different questions and listing the questions in it.

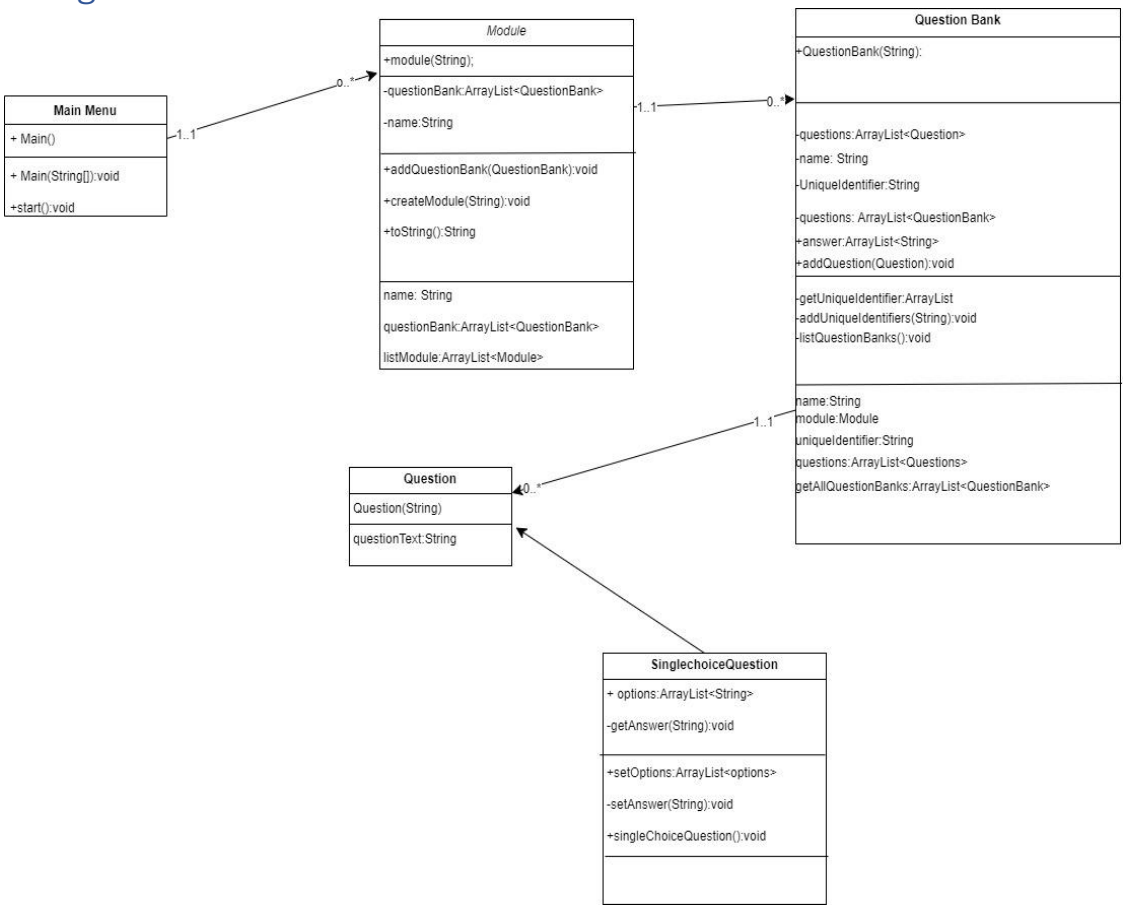
List Modules: this allows the display of every module created and helps prevent duplication of modules.

List Unique identifiers: it also displays every question bank created so the user is conscious about what question bank has been created and associated with a module, it also helps prevent the duplication of question banks in a module.

Creations Questions: this is the default and most important use case because everything else depends on it to work.

Take Quiz: this is only a part of the student's menu as they're the only ones able to take the quiz.

Design



Textual class description

Implementation of classes and how they interact with one another is crucial for this program as they work in hand to run inner communication.

Menu class: the menu class serves as an interface that houses the program. It allows Actors (Teacher and Student) to interact with the system by providing various options for different functionalism of the program.

Attributes:

- Question Banks attribute is an ArrayList that holds instances of the `QuestionBank` class, representing the aggregate of available question banks within the system.
- questionBank : this is an instance of QuestionBank class representing the current question Banks is being operated on.
- Unique Identifiers List: this is an ArrayList that stores unique identifiers associated with question banks.

Methods:

- Main (String[]args): This is the main method of the program, it initializes an instance of the Menu class and invokes the **Main Menu** method.
- main Menu(): method showcases the choices (teacher or student) to the user and requests a response. it directs the user to the relevant menu based on their input.

- teacherMenu (): this displays the menu options to the teacher, allowing the teacher to perform a specific task such as creating modules, Question banks, etc. It continuously loops until the user chooses to exit the menu.
- createModule() : Allows the teacher to create different modules. Check if the teacher's input isn't more than seven characters as requested in the requirement and check for duplicate module names.
- listModule() : this method helps display the list of current modules available in the system.
- createQuestionBannk() : Guides the teacher through the process of creating a new question bank and associating it with module. It also loops to check for existing question banks to ensure there are no duplicate question banks within the same module.
- ListQuestionBank (): it lists particularly the question banks associated with a module.
- ListUniqueIdentifier (): it lists Unique identifiers associated with Question banks.
- SingleChoiceQuestion (): Guides teacher through each step of creating a singlechoice question and adding it to a specific question bank.
- listQuestion() : lists all the questions available in a question bank.

Pseudocodes

```
function createQuestionBank():

  // Prompting the user to enter the name of the question bank

  display "Enter the name of your question bank".

  questionBankName = read input from user and stores it in QB name. ef


  // Check if the length of the question bank name is within the allowed limit

  if length of questionBankName is less than or equal to 15:

    display "Correct input".

  else:

    display "Enter the correct format".


  // Print existing modules for selection

  display "Here are the list of existing modules:"

  for each module in Module.modules:

    display module's name


  // Prompt the user to select a module to associate with the question bank

  display "Kindly select a module you want to associate with the QB"

  moduleName = read input from user
```

```
// Check if the selected module exists
selectedModule = null
for each module in Module.modules:
   if moduleName is equal to module's name:
      selectedModule = module
      break

// Notify the user if the selected module does not exist
if selectedModule is null:
   display "Module does not exist"
   return

// Check if a question bank with the same name already exists in the selected module
questionBankExists = false
for each questionBank in selectedModule's question banks:
   if questionBank's name is equal to questionBankName:
      questionBankExists = true
      break

// Notify the user if a question bank with the same name already exists in the selected module
if questionBankExists:
   display "Question bank already exists in the selected module”.
   return

// Create a new question bank and associate it with the selected module
newQuestionBank = create QuestionBank with name questionBankName
add newQuestionBank to selectedModule's question banks
display "Question bank has been created and associated with module " + selectedModule's name
```

pseudocode 2

```
function deleteQuestionBank()
   // Prompting user to enter the name of the module that houses the question bank they want to delete
   input moduleName from user

   // Search for the module to delete the question bank from
   moduleToDeleteFrom = findModuleByName(moduleName)

   // If module not found, display error message and return
   if moduleToDeleteFrom is null
      print "Module moduleName not found."
      return

   // Prompt user to enter the name of the question bank
   input questionBankName from user

   // Search for the question bank within the module
   questionBankToDelete = findQuestionBankByName(moduleToDeleteFrom, questionBankName)

   // If question bank not found, display error message and return
   if questionBankToDelete is null
      print "Question bank questionBankName not found in module moduleName."
      return

   // If question bank is empty, delete it from the module
   if isEmpty(questionBankToDelete)
      delete questionBankToDelete from moduleToDeleteFrom's questionBanks
      print "Question Bank questionBankName has been deleted."
   else
      print "Cannot delete question bank because it is not empty."
```

```
// Function to find a module by name
function findModuleByName(moduleName)
   for each module in Module.modules
      if module's name equals moduleName
         return module
   return null

// Function to find a question bank by name within a module
function findQuestionBankByName(module, questionBankName)
   for each questionBank in module's questionBanks
      if questionBank's name equals questionBankName
         return questionBank
   return null

// Function to check if a question bank is empty
function isEmpty(questionBank)
   return questionBank's questions list is empty
```

Test Tables

ID	REQUIREMENT	DESCRIPTION	INPUT	EXPECTED OUTPUT	PASS/FAIL	Comments
A1	FR1	A teacher creating empty question bank and associating it with one module ID	Name less than or equal 15 characters	Successfully created	pass	it ensures the input name meets the specified requirement
B1	FR2	A teacher can add new Single choice question to an existing question bank. 1.Taking question text 2. Taking number of options	Question: what is your name? 2 options.	Successfully adds a new single choice question to an existing	pass	

		3. Indicates which option is correct.	Correct answer: alim	question bank.		
B2	FR2B	Fill in the black question type			Fail	Programmed was developed with single choice question only
C1	FR3	A teacher can list and remove question from existing question bank	Number (7) on the teacher's menu.	Lists all the question in a question bank.	pass	
D1	FR4	A teacher can list all question bank for a specific module	Question bank that exists in the module	It displays it	pass	
			Question bank not in the module	To send error message	pass	It tells the question bank doesn't exist in that module
E1	FR5	A teacher can delete question bank only when its empty.	Number (8) on the teacher's menu	To delete question bank if it is empty	pass	
			Number (8) on the teacher's menu but with a question bank that doesn't exist	To send out error massage saying question bank doesn't exist	pass	It loops through the array list of question bank and makes sure it is empty before being able to delete it.
F1	FR6a	List all question bank for a specific module identifier: if module identifier is unknown then an error message should be displayed	Number (4) on the teacher's menu and inputting a module that exists	Lists out all the question bank for a module.	pass	

			Number (4) on the teacher's menu but with an input of a module that doesn't exist	Should print error message saying module not found	pass	If module id isn't recognised, then it prints an error message
F2	FR6B1	Take a specific question bank using its question bank unique id	Number (1) on the student's menu and inputting a bank id that exists	Should print out the question	pass	
			Number (1) on the student's menu and inputting a bank id that doesn't exists	Should print out an error message saying id not found.	pass	Prints out error message
G1	FR8	The quiz displays Q questions in a random order. It indicates the number of questions to be displayed. Only one question at a time is displayed and the question number is clearly indicated			fail	
H1	FR9	The student decides when they want to move to the next question or go back to a previous question.			fail	
J1	FR10	A single choice question is displayed to the student in an appropriate textual format allowing the student to choose an answer depending on the question type. For a fill-the-blanks question the student adds the missing words or	A student chooses correct answer for the single choice question.	Tells if the answer is Correct or not	pass	

		phrases for the blanks. The form this takes is up to you				
			Fill in the gap		fail	

Nonfunctional Requirement

ID	REQUIREMENT	Description	input	Expected output	Pass/fail	comment
A1	NFR1	The question bank must be persistent and stored in a text-based database: i.e. text files.			fail	
B1	NFR2	A text-based, menu-driven user interface.			pass	A text-based menu driven user interface is applied.
B3	NFR3	It must be possible to plug in new kinds of question in the future, hence the need for inheritance.			pass	The question class allows for inheritance which gives opportunity to allow all sort of question in the future.

SCREENSHOTS

Figure 2creating module.

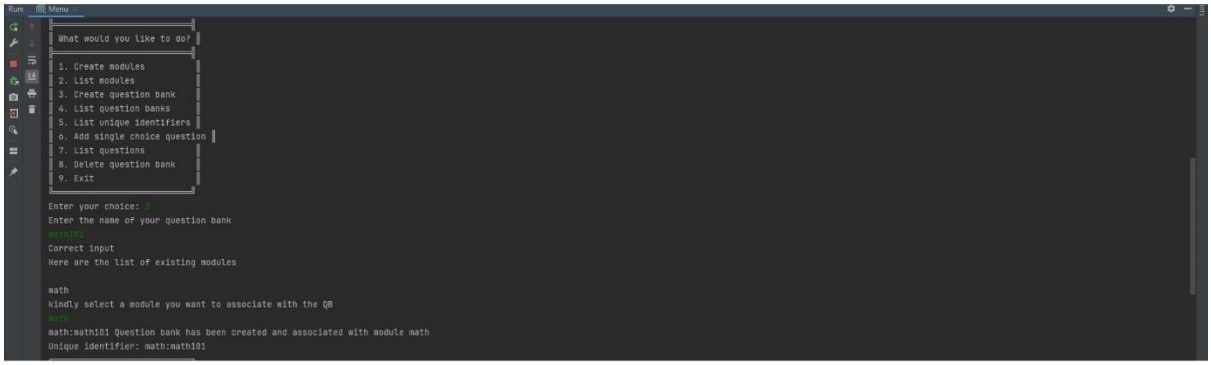


Figure 3Creating Question Bank

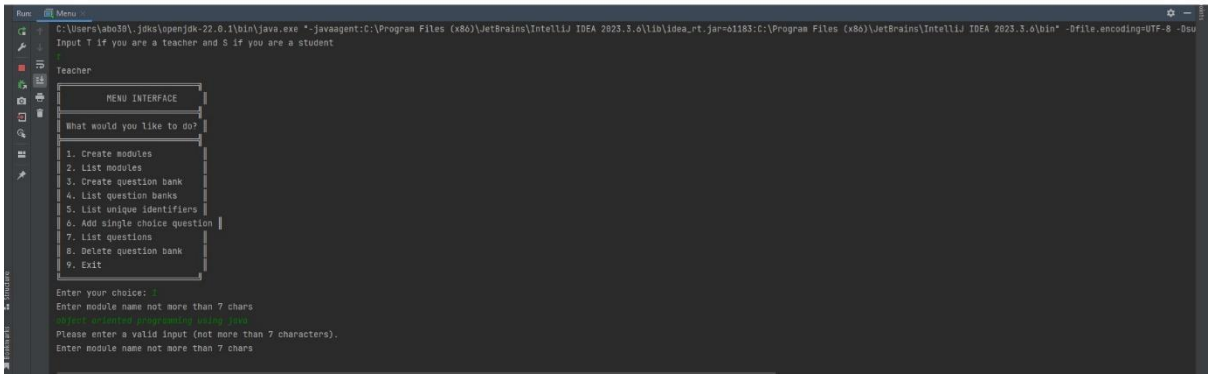


Figure 4 Capturing Wrong Inputs

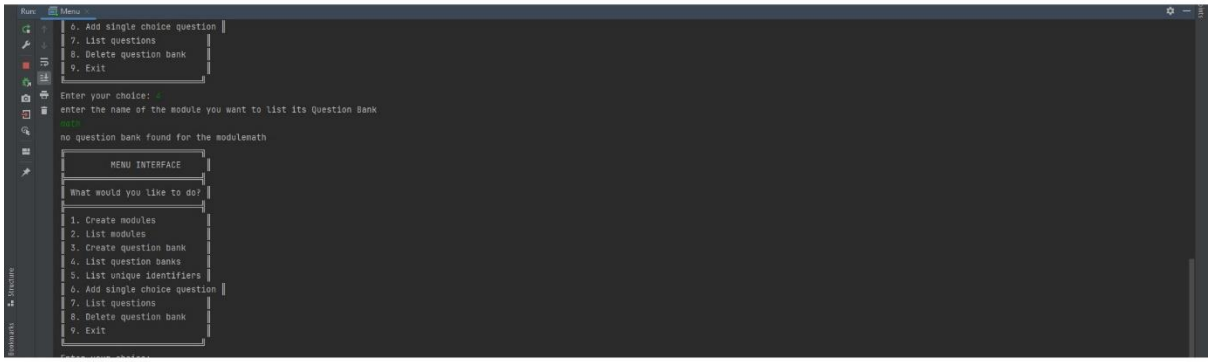


Figure 5 Capturing invalid Inputs.

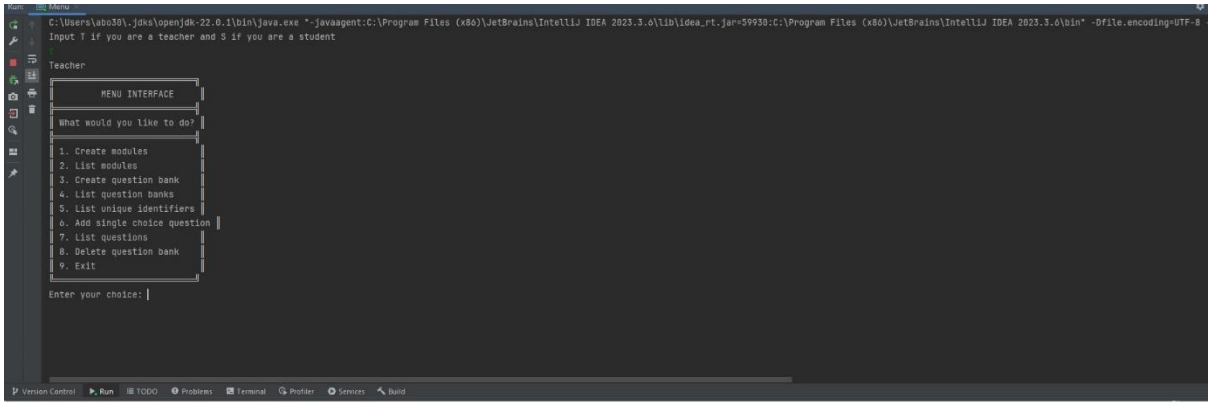


Figure 6 Teacher's menu interface

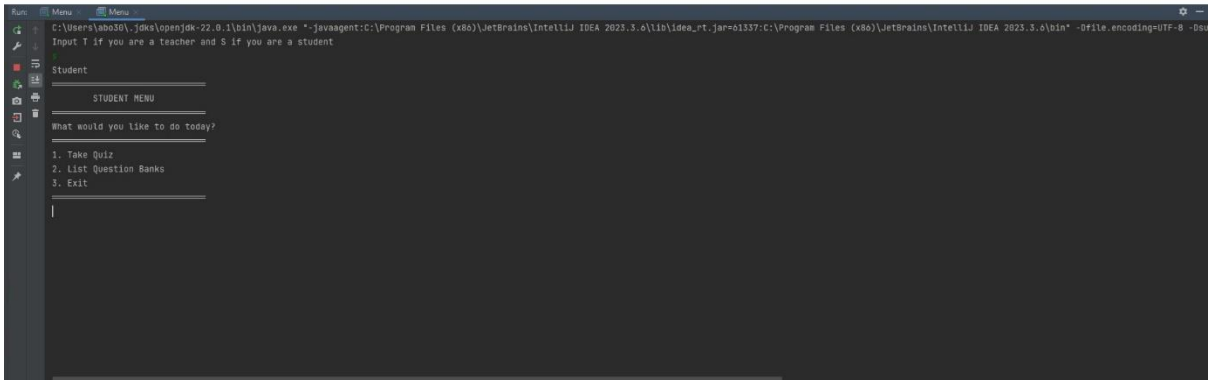


Figure 7 Student Menu Interface

EVALUATION

Some of the primary functions in this program demanded a lot of attention, with some taking weeks to fully comprehend. For instance, when adding a question to a question bank, I had to ensure the presence of a specific question bank and module as intended by the teacher to ensure validity. This was necessary for the development of a logic to iterate through the locations where both the module and question banks are stored, to pinpoint the exact module intended by the teacher.

The teacher's menu interface operates smoothly, employing various intertwined logics that communicate effectively to facilitate swift storage and retrieval of data as needed. Ensuring a welcoming and easily understandable interface is essential to leave a positive impression on the user and foster an enjoyable experience with the program. Furthermore, implementing measures to guide users on the required input types and displaying relevant warning messages can further enhance user experience and prevent errors.

The student method, however, remains somewhat underdeveloped, it offers limited operational options. While it fulfils the basic functional requirement of facilitating quizzes, its capabilities are restricted to handling single-choice questions exclusively, without the inclusion of fill-in-the-gaps questions. Furthermore, students are granted access to a list of question banks, enabling them to familiarize themselves with the unique identifiers associated with the questions available to them. This feature aims to avoid errors during quiz input by providing clarity on the available question options.

WHAT WAS DIFFICULT

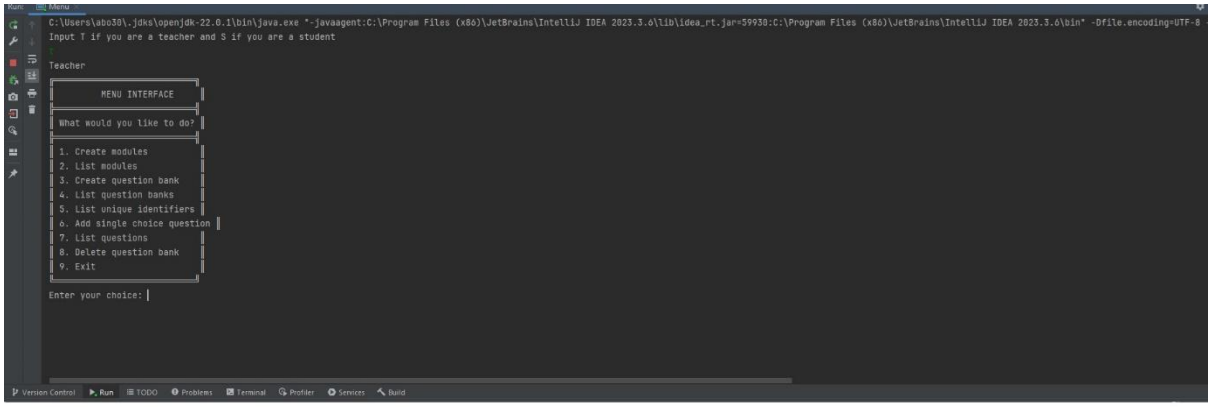
The most challenging aspect of developing this program was implementing the creation of question banks. This involved prompting the user to input a name for the question bank and specifying the module they wished to associate it with. Subsequently, it was necessary to verify that a question bank with the provided name did not already exist within that module, even if there were differences in capitalization. Finally, the program had to create the question bank and store it in an Array List. This process required careful handling to ensure data integrity and user-friendly interaction.

WHAT REMAINS TO BE DONE

Remaining to be addressed is the non-functional requirement regarding storing and retrieving data from a text file, which currently isn't functioning as expected. Some code related to this requirement was left in the program in the hope of receiving partial credit, even if it doesn't fully operate as intended. I dedicated considerable effort to researching how to properly append and read from a file but encountered challenges in implementing it effectively. Given more time, I would have been able to resolve these issues. Even after submission, I am committed to further refining the program to ensure it fully meets the specified requirements.

FLAIR

I explored online resources to learn how to utilize ASCII symbols effectively in crafting visually appealing menu interfaces for both students and teachers. This involved employing symbols to construct aesthetically pleasing designs that provide users with a warm and inviting welcome to the program.[1]



Moreover, I integrated a method into the teacher's menu called "List Unique Identifiers." True to its name, this function iterates through each saved question bank associated with a module, treating them as distinct identifiers. This aids teachers in identifying which question banks have been created in each module without the need to individually open every module. Such functionality enhances accessibility and help ease the user experience.

MARK

Most functional requirements are operating smoothly without errors, complemented by comprehensive documentation that thoroughly includes the workings of complex methods. Additionally, the inclusion of detailed comments in understanding each line of code. Having fulfilled most of the specified criteria, I assess my performance at approximately 75% to 80%, considering the accomplishments made.

References

[1] https://www.asciiart.eu/ascii-one-line

[2]https://app.diagrams.net/

[3]https://www.youtube.com/watch?v=hgF21imQ_Is